Multi-Agent AI System for Interactive Chemistry Learning and Simulation

Problem Statement

Current virtual chemistry learning platforms are limited in their ability to provide meaningful and interactive educational experiences. Most existing systems operate as static, rule-based simulators that follow predefined instructions without understanding user intent or adapting to individual learning needs.

While these platforms allow users to perform virtual experiments, they lack intelligent guidance, contextual feedback, and clear explanations of underlying chemical principles. As a result, students often follow step-by-step procedures without truly understanding why reactions occur, what concepts are involved, or how to correct mistakes.

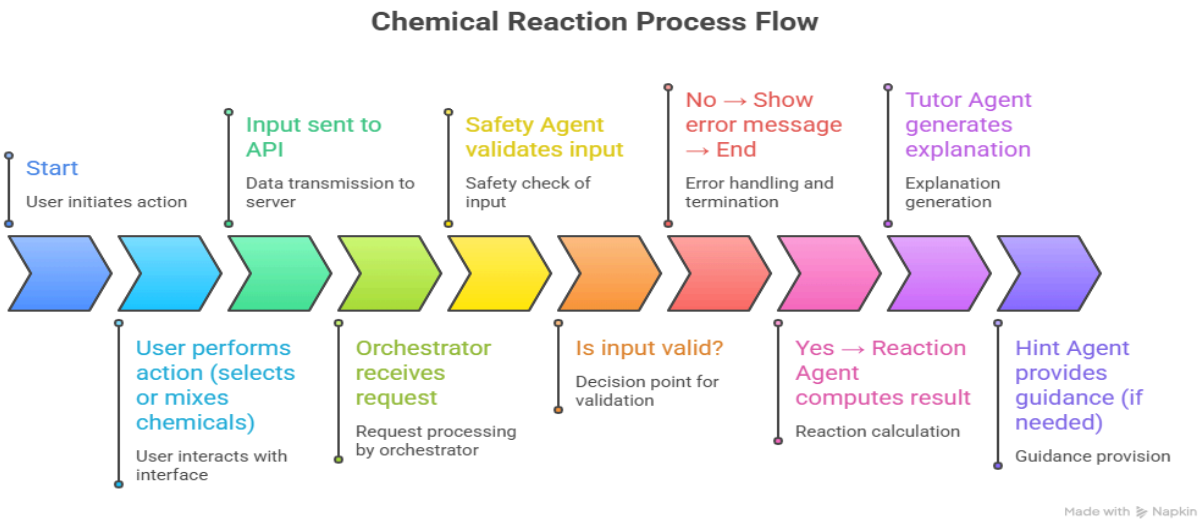
Additionally, error handling in these systems is often generic and uninformative, offering little support when users make incorrect choices. There is also minimal personalization, meaning all learners receive the same experience regardless of their skill level or progress.

This creates a “black box” learning environment where experimentation is possible, but conceptual understanding and engagement remain limited. Addressing these gaps requires a system that can simulate experiments accurately while also providing adaptive guidance, explanation, and feedback similar to a real laboratory instructor.

Proposed Solution

To address the limitations of traditional virtual labs, this project proposes a multi-agent AI system that simulates, validates, and explains chemistry experiments through a structured and collaborative architecture.

The system is designed around a central orchestration layer that coordinates multiple specialized AI agents, each responsible for a specific task.



Step 1: User Interaction Layer

Minimum Viable Product (MVP) Scope

To ensure successful completion within the summer camp timeline, the project will focus on a well-defined MVP:

Chemical Coverage:

- 15 common chemicals (e.g., HCl, NaOH, H_2SO_4, NaCl, $AgNO_3$, $CuSO_4$, $CaCO_3$, etc.)
- 4 reaction types: acid-base neutralization, precipitation, single displacement, and decomposition
- Approximately 30-40 validated reaction outcomes

Agent Capabilities:

- Safety Agent: Rule-based validation covering incompatible combinations and basic safety constraints
- Reaction Agent: Pre-computed reaction database with lookup functionality
- Tutor Agent: Template-based explanations with natural language generation for supported reactions
- Hint Agent: Contextual suggestions based on common mistake patterns

- Progress Agent: Simple skill tracking (beginner/intermediate/advanced)

This focused scope allows for a complete, functional system while leaving room for future expansion beyond the summer camp.

Step 2: Central Orchestrator

Experiment State Management:

The Central Orchestrator maintains a persistent state object that tracks:

- Inventory: All chemicals currently available in the virtual lab bench
- Sequence: Chronological log of user actions (mixes, measurements, heating, etc.)
- Active experiment: Current mixture composition and concentrations
- Session history: Previous reactions performed during the session

This state is stored in a JSON structure that is updated after each validated action. The state enables multi-step experiments (e.g., performing a reaction, then testing the product with another chemical) and allows the Progress Agent to evaluate sequential reasoning. If the system crashes, the state can be serialized to preserve the user's progress.

Step 3: Safety Validation

Safety Agent Implementation Strategy:

To ensure reliability, the Safety Agent will use a three-layer validation approach:

1. Explicit rule base: A curated list of forbidden combinations (e.g., concentrated acids with water, reactive metals with strong acids) derived from standard chemistry lab safety guidelines
2. Reaction type validation: Verification that the proposed reaction follows known chemical behavior patterns (e.g., acid + base → salt + water)

3. Confidence scoring: If a reaction is not explicitly in the knowledge base but is chemically plausible, the system returns a "low confidence" response and prompts user confirmation

All safety rules will be reviewed by a chemistry educator before deployment, and the system will default to "safe stop" (refuse to execute and explain why) when uncertain.

Step 4: Reaction Computation

LLM Integration Strategy:

To manage cost and ensure chemical accuracy, the system will use a hybrid approach:

Component	Approach	Rationale
Tutor Agent	**Template-based with LLM augmentation**	Core explanations use pre-written templates for supported reactions. LLM (local or API) is only invoked for novel user questions, with chemical context provided in the prompt to ensure accuracy.
Hint Agent	**Rule-based with LLM fallback**	Hints are generated from a pre-defined library of common mistakes. LLM is used only if no matching hint exists to provide adaptive assistance.

Safety Agent	**Rule-based only**	No LLM involvement to prevent unsafe suggestions. This ensures 100% deterministic validation of hazardous or restricted reactions.

Cost Mitigation:

- Use open-source local models (e.g., Llama 3, Mistral) where possible to eliminate API costs
- If using cloud LLMs, implement caching to avoid repeated identical queries
- Set daily usage limits and monitor costs during development

Accuracy Assurance:

- All LLM-generated explanations are post-processed to verify chemical formulas and reaction equations against the Reaction Agent's validated output
- Disclaimers indicate when content is AI-generated
- A "Report Issue" button allows users to flag inaccurate explanations for review

---

Step 5: Explanation Generation

The Tutor Agent converts the reaction result into a clear explanation.

Features:

- Step-by-step reasoning
- Concept-based explanation (e.g., neutralization)
- Optional guided questions for deeper understanding

Step 6: Adaptive Assistance

If the user struggles or takes incorrect steps, the Hint Agent provides contextual guidance.

Types of hints:

- Subtle suggestions
- Step hints
- Direct instructions (if needed)

Step 7: Progress Tracking

The Progress Agent updates the user's learning profile.

Tracks:

- Completed concepts
- Mistakes and patterns
- Skill level

This enables adaptive difficulty and personalized learning.

Example System Output

This JSON shows how the system processes a user experiment step-by-step. The user inputs two chemicals, which are first validated by the Safety Agent. If valid, the Reaction Agent computes the result. The Tutor Agent then explains the reaction, and the Progress Agent updates the user's learning status. This demonstrates how multiple agents work together to provide both accurate simulation and meaningful learning.


```
{
  "input": {
    "chemical_1": "HCl",
    "chemical_2": "NaOH"
  },
  "validation": {
    "status": "valid",
    "checked_by": "Safety Agent"
  },
  "reaction": {
    "type": "Neutralization",
    "products": ["NaCl", "H2O"],
    "energy_change": "-57 kJ/mol"
  },
  "explanation": {
    "agent": "Tutor Agent",
    "text": "This is a neutralization reaction where an acid reacts with
    a base to form salt and water."
  },
  "progress": {
    "concept": "Neutralization",
    "status": "completed"
  }
}
```

System Architecture

User → API → Orchestrator → Agents

Agents:

- Reaction Agent
- Safety Agent
- Tutor Agent
- Hint Agent
- Progress Agent

The orchestrator ensures all agents communicate efficiently and operate as a unified system.

Alignment with Modular Framework of Agent(MOFA) Goals:

This project is designed to align with MOFA's focus on modular, reusable, and extensible agent frameworks.

---

1. Clean Modular Architecture

- Each agent has a single, well-defined responsibility
- Safety, Reaction, Tutor, Hint, and Progress agents are independently developed and maintained
- Agents can be replaced, upgraded, or reused without affecting others

2. Standardized Agent Communication

- All agents communicate through a unified protocol via the Central Orchestrator
- Clear input/output contracts enable interoperability and testability
- Example format: JSON-based requests with agent, action, payload, and context fields

3. Reusable Framework Patterns

- State Management: JSON-based persistent state for multi-step workflows

- Hybrid Agent Pattern: Combines deterministic rules with LLM-based agents
- Orchestrator Pattern: Central coordination with extensible agent routing
- Safety-First Validation: Three-layer validation applicable to other domains

4. Open Source Deliverables

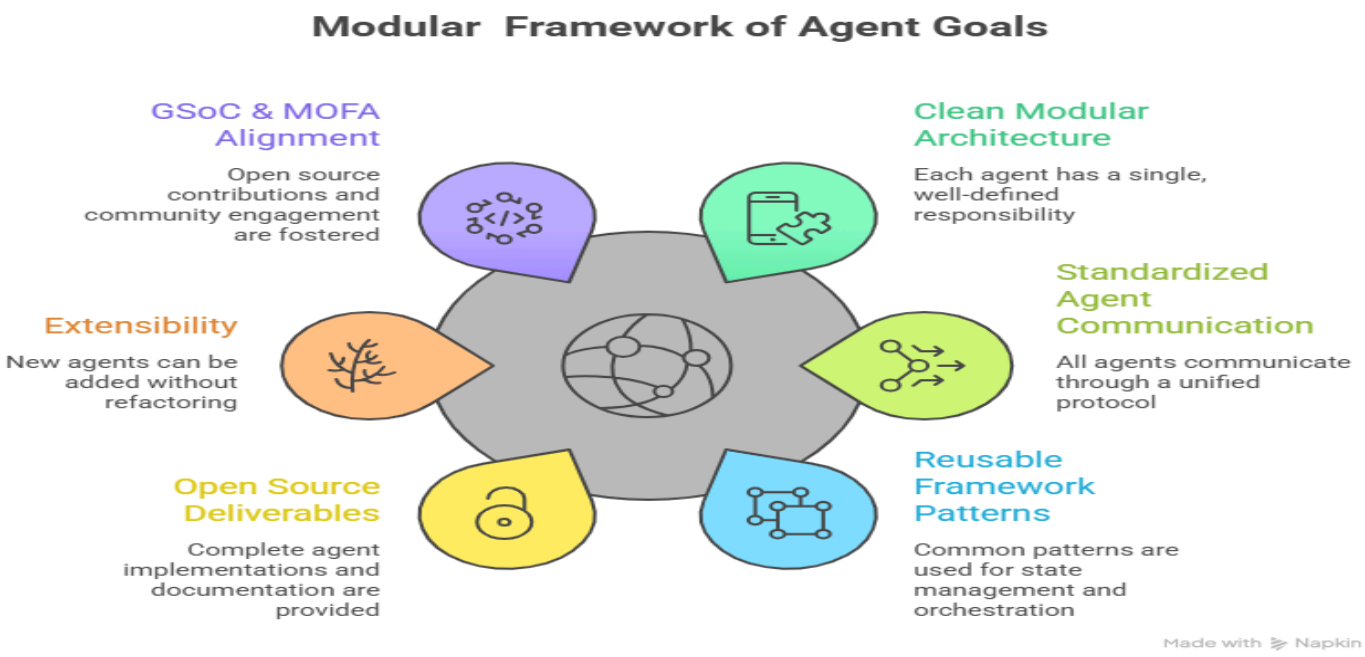
- Complete agent implementations with documentation
- Reference architecture for educational multi-agent systems
- Agent communication schemas reusable across projects
- Template knowledge base for domain adaptation

5. Extensibility

- New agents (Visualization, Quiz, Voice Assistant) can be added without refactoring
- Architecture supports future expansion to other science domains (physics, biology)

6. GSoC & MOFA Alignment

- Open source contribution to MOFA repositories
- Community engagement through mentor collaboration
- Sustainable long-term value through reusable patterns



Conclusion: This project delivers a modular, extensible multi-agent system that serves as a practical reference implementation for the MOFA community.

Workflow Summary

1. User performs an action
2. Input sent to orchestrator
3. Safety Agent validates input
4. Reaction Agent computes result
5. Tutor Agent explains outcome
6. Hint Agent assists if needed
7. Progress Agent updates learning state

This structured approach ensures that the system is accurate, adaptive, and capable of providing a learning experience similar to a real laboratory instructor.